Review of Previous Meeting

- **To-Do Items**
  - Revise glitch descriptions to be **self-contained (standalone)**.
  - Adopt a **hierarchical taxonomy** for the glitch list.
  - Execute classification tests using the updated framework.

Experimental Results

- Perfectly Correct: 3 / 8 cases
- Partially Correct: 3 / 8 cases (identified correct parent categories)
- Incorrect: 2 / 8 cases

Failure Analysis 1: Environmental Bias

- **Case:** Ledge Clipping in Princess Peach's Castle misidentified as **LBLJ**.
- LBLJ typically occurs in the castle lobby and involves high-speed movement and wall clipping via BLJ.
  - The model likely prioritized **location-based cues** (Castle) over the specific physical mechanics of the glitch.

Ledge Clipping

LBLJ

Failure Analysis 2: Visual Occlusion

- **Case:** BLJ in Lethal Lava Land (LLL) misidentified as **Pole Glitch**.
  - Mario became temporarily invisible due to suboptimal camera work.
  - Both glitches occur in LLL; the frames included visual triggers for Pole Glitch (poles and metal grids) during the high-speed movement.
  - The model relied on these environmental cues instead of recognizing the motion context.

BLJ in LLL

Pole Glitch

Proposed Solutions

- Few-shot Learning: Utilize visual examples to define glitch characteristics (Priority 1).
- Multimodal Integration: Leverage models capable of processing auditory information (Priority 2).
- Temporal Context Extension: Increase sequence duration by lowering FPS to capture more motion context (Priority 3).

Methodology for Preliminary Few-shot Experiments

- Objective: Targeting the previously failed BLJ detection in Lethal Lava Land (LLL).
- Method: Providing visual examples (Few-shot) to the model to help it recognize the specific motion patterns of a BLJ.
- Setup: Sourced a GIF of a BLJ from a web database to serve as a reference for the model before it analyzes the test sequence.

BLJ in LLL

Example

Challenges with Few-shot Learning

- Result: ❌ failed
- **Comment 1:** If we can source a GIF of a BLJ specifically performed in LLL from the internet and present it as an example, the detection task might become significantly easier (potentially reducible to a simple pattern-matching task at the 3D ResNet level).
- **Comment 2:** Using clips from other records as examples is **self-defeating** because it requires humans to manually locate the glitches first. This contradicts our ultimate goal of automating the entire bug-finding process.

Image Count Limit

- **Comment 3:** OpenAI's API has a limit on the number of images per request (typically around 100). Considering our taxonomy consists of 19 glitch categories, this constraint is quite restrictive.

Leveraging Auditory Information

- Observation: Visual motion is often obscured by camera work (e.g., LLL BLJ).
- Hypothesis: Auditory cues, such as Mario’s vocal triggers ("Yahoo!"), provide vital evidence for specific actions.

- Conclusion: Character voices and sound effects are powerful features for disambiguating glitches.

Method 1: Native Video Input

- Approach: Inputting raw video files (e.g., mp4) directly into the model.
- Pros: Most ideal for preserving temporal and auditory synchronization.
- Models:
  - Gemini 1.5 Pro / Flash
  - GPT-4o

Method 2: Synchronized Frames & Audio Files

- Approach: Inputting image sequences and separate audio files (e.g., mp3) simultaneously.
- Assessment: A highly realistic and actionable approach for multimodal LLMs.
- Models:
  - GPT-4o
  - Video-LLaMA

Method 3: Frames & Audio Transcription

- Approach: Converting audio cues into text descriptions (e.g., "[Mario: Yahoo!]") to supplement frames.
- Pros: Highly token-efficient.
- Cons: Significant risk of losing fine-grained temporal precision, tone, and specific sound effect nuances.
- Models:
  - GPT-4o / GPT-4 Turbo
  - Claude 3.5 Sonnet

Key Question

- What is the optimal approach for integrating audio in terms of feasibility, cost, and classification accuracy?